



Hybrid Search with Vector DB

Import the Needed Packages

```
In [ ]: import warnings
warnings.filterwarnings('ignore')

In [ ]: from datasets import load_dataset
from pinecone_text.sparse import BM25Encoder
from pinecone import Pinecone, ServerlessSpec
from DLAIUtils import Utils

from sentence_transformers import SentenceTransformer
from tqdm.auto import tqdm
import torch
import os

In [ ]: utils = Utils()
PINECONE_API_KEY = utils.get_pinecone_api_key()
```

Setup Pinecone

```
In [ ]: device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(device)

utils = Utils()
INDEX_NAME = utils.create_dlai_index_name('dl-ai')

pinecone = Pinecone(api_key=PINECONE_API_KEY)

if INDEX_NAME in [index.name for index in pinecone.list_indexes()]:
    pinecone.delete_index(INDEX_NAME)
pinecone.create_index(
    INDEX_NAME,
    dimension=512,
    metric="dotproduct",
    spec=ServerlessSpec(cloud='aws', region='us-west-2')
)
index = pinecone.Index(INDEX_NAME)
```

Load the Dataset

```
In [ ]: fashion = load_dataset(
    "ashraq/fashion-product-images-small",
    split="train"
)
fashion
```

```
In [ ]: images = fashion['image']
        metadata = fashion.remove_columns('image')
        images[900]
```

```
In [ ]: metadata = metadata.to_pandas()
        metadata.head()
```

Create the Sparse Vector Using BM25

```
In [ ]: bm25 = BM25Encoder()
        bm25.fit(metadata['productDisplayName'])
        metadata['productDisplayName'][0]
```

```
In [ ]: bm25.encode_queries(metadata['productDisplayName'][0])
        bm25.encode_documents(metadata['productDisplayName'][0])
```

Create the Dense Vector Using CLIP

```
In [ ]: model = SentenceTransformer('sentence-transformers/clip-ViT-B-32',
        device=device)
        model
        dense_vec = model.encode([metadata['productDisplayName'][0]])
        dense_vec.shape
```

```
In [ ]: len(fashion)
```

Create Embeddings Using Sparse and Dense

(Note: fashion_data_num = 1000): In this lab, we've initially set `fashion_data_num` to 1000 for speedier results, allowing you to observe the outcomes faster. Once you've done an initial run, consider increasing this value. You'll likely notice better and more relevant results.

```
In [ ]: batch_size = 100
        fashion_data_num = 1000

        for i in tqdm(range(0, min(fashion_data_num, len(fashion)), batch_size)):
            # find end of batch
            i_end = min(i+batch_size, len(fashion))
            # extract metadata batch
            meta_batch = metadata.iloc[i:i_end]
            meta_dict = meta_batch.to_dict(orient="records")
            # concatenate all metadata field except for id and year to form a single s
            meta_batch = [" ".join(x) for x in meta_batch.loc[:, ~meta_batch.columns.i
            # extract image batch
            img_batch = images[i:i_end]
```

```

# create sparse BM25 vectors
sparse_embeds = bm25.encode_documents([text for text in meta_batch])
# create dense vectors
dense_embeds = model.encode(img_batch).tolist()
# create unique IDs
ids = [str(x) for x in range(i, i_end)]

upserts = []
# loop through the data and create dictionaries for uploading documents to
for _id, sparse, dense, meta in zip(ids, sparse_embeds, dense_embeds, meta_batch):
    upserts.append({
        'id': _id,
        'sparse_values': sparse,
        'values': dense,
        'metadata': meta
    })
# upload the documents to the new hybrid index
index.upsert(upserts)

# show index description after uploading the documents
index.describe_index_stats()

```

Run Your Query

```

In [ ]: query = "dark blue french connection jeans for men"

sparse = bm25.encode_queries(query)
dense = model.encode(query).tolist()

result = index.query(
    top_k=14,
    vector=dense,
    sparse_vector=sparse,
    include_metadata=True
)

imgs = [images[int(r["id"])] for r in result["matches"]]
imgs

```

```

In [ ]: from IPython.core.display import HTML
from io import BytesIO
from base64 import b64encode

# function to display product images
def display_result(image_batch):
    figures = []
    for img in image_batch:
        b = BytesIO()
        img.save(b, format='png')
        figures.append(f'''
            <figure style="margin: 5px !important;">
                
        {''.join(figures)}
        </div>
    ''')

```

```
In [ ]: display_result(imgs)
```

Scaling the Hybrid Search

```
In [ ]: def hybrid_scale(dense, sparse, alpha: float):
    """Hybrid vector scaling using a convex combination

    alpha * dense + (1 - alpha) * sparse

    Args:
        dense: Array of floats representing
        sparse: a dict of `indices` and `values`
        alpha: float between 0 and 1 where 0 == sparse only
               and 1 == dense only
    """
    if alpha < 0 or alpha > 1:
        raise ValueError("Alpha must be between 0 and 1")
    # scale sparse and dense vectors to create hybrid search vecs
    hsparse = {
        'indices': sparse['indices'],
        'values': [v * (1 - alpha) for v in sparse['values']]
    }
    hdense = [v * alpha for v in dense]
    return hdense, hsparse

```

1. More Dense

```
In [ ]: question = "dark blue french connection jeans for men"
        #Closer to 0==more sparse, closer to 1==more dense
        hdense, hsparse = hybrid_scale(dense, sparse, alpha=1)
        result = index.query(
            top_k=6,
            vector=hdense,
            sparse_vector=hsparse,
            include_metadata=True
        )
        imgs = [images[int(r["id"])] for r in result["matches"]]
        display_result(imgs)

```

```
In [ ]: for x in result["matches"]:
        print(x["metadata"]["productName"])

```

2. More Sparse

```
In [ ]: question = "dark blue french connection jeans for men"
        #Closer to 0==more sparse, closer to 1==more dense
        hdense, hsparse = hybrid_scale(dense, sparse, alpha=0)
        result = index.query(
            top_k=6,
            vector=hdense,
            sparse_vector=hsparse,
            include_metadata=True
        )
        imgs = [images[int(r["id"])] for r in result["matches"]]
        display_result(imgs)
```

```
In [ ]: for x in result["matches"]:
        print(x["metadata"]['productDisplayName'])
```

More Dense or More Sparse?

```
In [ ]: question = "dark blue french connection jeans for men"
        #Closer to 0==more sparse, closer to 1==more dense
        hdense, hsparse = hybrid_scale(dense, sparse, alpha=1)
        result = index.query(
            top_k=6,
            vector=hdense,
            sparse_vector=hsparse,
            include_metadata=True
        )
        imgs = [images[int(r["id"])] for r in result["matches"]]
        display_result(imgs)
```

```
In [ ]: for x in result["matches"]:
        print(x["metadata"]['productDisplayName'])
```